

Odesa I. I. Mechnikov National University
Faculty of Mathematics, Physics and Information Technologies
Department of Optimal Control and Economic Cybernetics

COURSEWORK

Bachelor's degree

on the topic: **“Fraud Detection in the Ethereum Network
with Machine Learning Methods”**

Author: third-year student
of the full-time programme
speciality 113 Applied Mathematics
Uliana O. Hrab

Supervisor: Cand. Sc. (Phys.-Math.), As-
soc. Prof. Ye. M. Strakhov

CONTENTS

Introduction	3
1 Theoretical foundations of fraud detection	5
1.1 Problem statement and domain	5
1.2 Binary classification methods	6
1.2.1 Logistic regression	6
1.2.2 Decision trees and random forest	6
1.2.3 Gradient boosting	7
1.3 Class imbalance	8
1.4 Classification quality metrics	8
1.5 Data leakage and correct evaluation	9
2 Implementation and analysis of the results	11
2.1 Description of the dataset	11
2.2 Exploratory data analysis	12
2.3 Duplicates and data leakage	15
2.4 Feature engineering and preprocessing	16
2.5 Model training and selection of the best model	17
2.6 Test-set results and their analysis	19
Conclusions	23
Bibliography	25
Appendix A. Implementation of the grouped split	27

INTRODUCTION

Over the past decade blockchain technology, and the Ethereum network in particular, has grown into one of the largest financial infrastructures in the world. Smart-contract support turned Ethereum into a platform for thousands of decentralised-finance applications, token exchanges and digital-asset issuers. The flip side of that growth is the spread of fraud: phishing schemes, Ponzi schemes, fake tokens and money laundering. Because accounts on the network are pseudonymous and transactions run into the millions per day, detecting malicious activity by hand is not practical. At the same time the entire transaction history is public, so it can be turned into behavioural features of accounts and fed to machine-learning methods for automatic classification [15].

Detecting fraudulent accounts is usually stated as a binary classification problem: given a set of features computed from an account’s transaction history, assign the account to one of two classes — legitimate or fraudulent. That is the formulation used here. The models are trained on an open dataset built from real Ethereum addresses, in which each account is described by fifty features of its transaction activity.

Particular attention is paid to evaluating model quality correctly. In problems of this kind high metric values often turn out to be inflated by hidden data-quality problems — above all by duplicates and data leakage, where at evaluation time the model is effectively handed rows it has already seen during training. Splitting the data properly and choosing appropriate metrics therefore matter as much as the choice of algorithm.

The interest in the topic comes from the intersection of machine learning and finance, in particular the financial technologies that have developed rapidly in recent years. Blockchain fraud is a practical and growing problem, and its data suits machine-learning methods well. This made it possible to work on a current problem and at the same time deepen applied skills while getting a closer look at how a blockchain works.

The aim of the work is to develop and compare machine-learning models for automatically detecting fraudulent Ethereum accounts from features of their transaction activity, while ensuring that their quality is evaluated correctly and

without data leakage.

The object of study is the process of detecting fraudulent account activity in the Ethereum network.

The subject of study is machine-learning methods and models for binary classification of Ethereum accounts by features of their transaction activity.

To achieve this aim the following **tasks** were set:

- 1) analyse the problem domain and the available dataset;
- 2) review binary classification methods and quality metrics suitable for imbalanced data;
- 3) carry out exploratory data analysis and identify data-quality problems;
- 4) engineer features and build a preprocessing pipeline;
- 5) train and compare several models and select the best one by cross-validation;
- 6) analyse the results obtained and draw conclusions.

Research methods: machine-learning methods (logistic regression, random forest, gradient boosting), exploratory and statistical data analysis, cross-validation, binary classification quality metrics.

The practical value of the work is that the resulting fraud-detection pipeline, and in particular the proposed way of splitting the data correctly, can serve as a basis for systems that monitor suspicious activity in blockchain networks.

The software part is implemented in Python using the scikit-learn, XGBoost, LightGBM, pandas and matplotlib libraries.

The work consists of an introduction, two chapters, conclusions, a list of references and an appendix. The first chapter sets out the theory: the problem statement, a review of classification methods, class imbalance, quality metrics and the data-leakage problem. The second chapter is devoted to the practical part — data analysis, feature engineering, model training and analysis of the results.

CHAPTER 1

THEORETICAL FOUNDATIONS OF FRAUD DETECTION

1.1 Problem statement and domain

Ethereum is a decentralised blockchain platform in which the state of the system is changed by transactions between accounts [4]. Each account is identified by an address, and its entire transaction history is stored publicly. This makes it possible to compute, for any address, a set of numeric characteristics of its behaviour: the number of transactions sent and received, the volumes of ether transferred, the time intervals between operations, the number of unique counterparties, activity with ERC20 tokens, and so on. These characteristics serve as the features for machine learning [8].

Some accounts are fraudulent — linked to phishing, Ponzi schemes, fake tokens or money laundering. The task is to tell a fraudulent account from a legitimate one automatically, using behavioural features of the account. Formally this is a binary classification problem.

Let a training sample be given:

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \{0, 1\}, \quad (1.1)$$

where $\mathbf{x}_i$ is the vector of d features of the i -th account and y_i is the class label ($y_i = 1$ for a fraudulent account, $y_i = 0$ for a legitimate one). The goal is to build a model that estimates the conditional probability of belonging to the fraud class,

$$\hat{p}(\mathbf{x}) = \hat{P}(y = 1 \mid \mathbf{x}), \quad (1.2)$$

and makes a decision from it using a threshold t :

$$\hat{y}(\mathbf{x}) = \begin{cases} 1, & \hat{p}(\mathbf{x}) \geq t, \\ 0, & \hat{p}(\mathbf{x}) < t. \end{cases} \quad (1.3)$$

The model parameters are chosen so as to minimise the error on the training data, and its quality is checked on a separate test sample that was not used during training.

1.2 Binary classification methods

Four models are compared in this work: logistic regression as a simple linear baseline, and three tree-based ensemble methods — random forest and two variants of gradient boosting (XGBoost and LightGBM) [16].

1.2.1 Logistic regression

Logistic regression is a linear classification model that estimates the class probability as a logistic function of a linear combination of the features:

$$\hat{p}(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b), \quad \sigma(z) = \frac{1}{1 + e^{-z}}, \quad (1.4)$$

where $\mathbf{w}$ is the weight vector, b is the intercept, and $\sigma(z)$ is the sigmoid, which maps the real line onto the interval $(0, 1)$. The parameters are found by minimising the logistic loss (cross-entropy) with regularisation:

$$L(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln \hat{p}(\mathbf{x}_i) + (1 - y_i) \ln (1 - \hat{p}(\mathbf{x}_i)) \right] + \lambda \|\mathbf{w}\|^2. \quad (1.5)$$

The advantages of logistic regression are simplicity and interpretability: the sign and magnitude of a weight w_j show the direction and strength of a feature's influence. Its limitation is the linearity of the decision boundary: it captures complex non-linear relationships between features poorly [10]. It is therefore used here mainly as a baseline for comparison.

1.2.2 Decision trees and random forest

A decision tree partitions the feature space into regions by successive splits on threshold values of individual features. The quality of a split is judged

by the reduction in node impurity; for classification a common measure is the Gini index:

$$G = 1 - \sum_k p_k^2, \quad (1.6)$$

where p_k is the fraction of objects of class k in the node. Trees model non-linear dependencies and feature interactions naturally, but a single deep tree tends to overfit.

The random forest [3] overcomes this limitation by building an ensemble of many trees and averaging their predictions. Each tree is trained on a bootstrap sample of the training data (bagging), and at every node the split is searched for only among a random subset of features. This double randomness makes the trees weakly correlated, so averaging their predictions substantially reduces variance and improves the model’s ability to generalise.

1.2.3 Gradient boosting

Unlike a random forest, where trees are built independently, gradient boosting builds them sequentially, each new tree correcting the errors of the previous ones [9]. The model is formed as an additive sum:

$$F_m(\mathbf{x}) = F_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x}), \quad (1.7)$$

where h_m is the next tree, approximating the negative gradient of the loss function on the current residuals, and $\nu \in (0, 1]$ is the learning rate, which restrains the contribution of each tree and prevents overfitting.

Two modern implementations of gradient boosting are used here. **XG-Boost** [6] adds tree-complexity regularisation to the loss function and uses efficient split-finding algorithms. **LightGBM** [13] grows trees leaf-wise and bins feature values into histograms, which makes it train faster and scale better on large data. Tree ensembles, and gradient boosting in particular, usually give the best results on tabular data, so they are expected to perform best here as well.

1.3 Class imbalance

In fraud-detection problems the classes are almost always imbalanced: legitimate accounts far outnumber fraudulent ones. In the dataset used here the fraud share is about 22%. Under such imbalance plain accuracy becomes misleading: a trivial model that always predicts “legitimate” is right about 78% of the time without detecting a single fraudster.

To keep the model from ignoring the rare class, one of two approaches is used [11]. The first is resampling: increasing the number of minority-class objects or reducing the majority, for instance with the synthetic-generation method SMOTE [5]. The second is cost-sensitive learning through class weights: an error on the rare class is penalised more heavily, in proportion to its share. This work uses the second approach, since it does not distort the original data: class weighting (`class_weight = balanced`) is set for logistic regression and the random forest, the `scale_pos_weight` parameter for XGBoost, and the `is_unbalance` mode for LightGBM.

1.4 Classification quality metrics

The results of binary classification are conveniently presented as a confusion matrix, which holds the counts of correctly and incorrectly classified objects of each class: true positives TP , false positives FP , false negatives FN and true negatives TN [18]. From it the basic metrics are defined — precision and recall:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (1.8)$$

Precision shows what fraction of the accounts flagged as fraudulent really are fraudulent, and recall shows what fraction of all fraudsters the model found. These metrics trade off against each other, so they are often combined into the F_1 measure — their harmonic mean:

$$F_1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (1.9)$$

All of these metrics depend on the chosen threshold t . To assess a model

independently of the threshold, aggregate measures are used. The ROC curve plots the true-positive rate against the false-positive rate as the threshold varies, and the area under it (ROC-AUC) is an integral measure of quality. Under strong imbalance, however, ROC-AUC is too optimistic: the large number of true negatives keeps the false-positive rate low even for mediocre models [7].

More informative for imbalanced data is the precision–recall curve and the area under it (PR-AUC) [19]. It focuses precisely on the rare positive class and does not mask errors behind a large number of legitimate objects. The PR-AUC baseline for a random classifier equals the positive-class share (about 0.22), whereas for ROC-AUC it is always 0.5. For this reason PR-AUC is taken as the main metric in this work, with the remaining metrics treated as supplementary.

1.5 Data leakage and correct evaluation

Data leakage is a situation in which the model receives, during training, information that would not be available at real prediction time. The consequence is inflated quality estimates on the test set that do not reproduce on new data [12]. A distinction is drawn between target leakage, where the answer is present among the features directly or indirectly, and test-set contamination, where information from the test set seeps into training.

It is the second type that is relevant here. If the data contain duplicates — rows with identical feature values — then under a random split exact copies of the same account can end up in both the training and the test sample. The model is then evaluated on rows it has effectively already seen during training, and the metrics come out inflated not by the quality of the model but by its recognising familiar objects.

To avoid this, the split has to take groups of identical objects into account: all rows with an identical feature profile must go entirely into one sample only. The standard stratified scheme, which preserves the class ratio, is therefore replaced here by a grouped stratified one (StratifiedGroupKFold): it preserves the class share and at the same time prevents identical profiles from crossing the boundary between training and test.

Preprocessing is a separate source of leakage. If the parameters of the transformations (medians for imputing missing values, means and standard deviations for scaling) are computed on the whole dataset, statistics of the test sample seep into training. The preprocessing pipeline is therefore fitted on the training data only, and the already fitted transformation is applied to the test data. The theoretical apparatus described here — metrics, grouped cross-validation and leakage control — is applied directly in the practical part of the work.

CHAPTER 2

IMPLEMENTATION AND ANALYSIS OF THE RESULTS

2.1 Description of the dataset

The study uses an open Ethereum fraud-detection dataset published on Kaggle [2] and built from real addresses on the network [8]. It contains 9841 accounts, each described by fifty numeric features computed from its transaction history. The target feature **FLAG** takes the value 1 for a fraudulent account and 0 for a legitimate one.

The features fall roughly into four groups:

- operation counters — the number of transactions sent and received, contracts created, unique sender addresses and unique recipient addresses;
- time characteristics — the average interval between transactions and the span between the account's first and last operation;
- ether volumes — the maximum, average and total amounts of ether sent and received, and the resulting balance;
- ERC20 token activity — the number of such transactions and the types of the most frequently sent and received tokens.

The classes in the dataset are imbalanced: fraudulent accounts make up about 22% of the observations against 78% legitimate ones (Fig. 2.1). This imbalance drove the subsequent choice of metrics and model settings. All processing was done in Python using the pandas, scikit-learn, XGBoost and LightGBM libraries [14, 17].

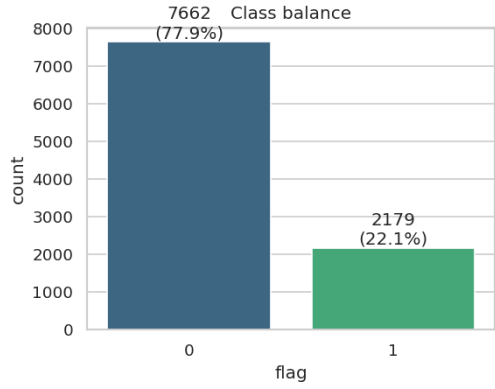


Figure 2.1. Ratio of legitimate to fraudulent accounts in the dataset

2.2 Exploratory data analysis

The purpose of the exploratory analysis was to assess data quality and to understand how well the features separate the classes, before any model was trained.

The distributions of the features related to ether volumes turned out to be strongly skewed: most accounts have small values, while a small fraction have values several orders of magnitude larger. For such features a logarithmic transform brings the distribution closer to symmetric and reduces the influence of outliers (Fig. 2.2). This observation was used later during feature engineering.

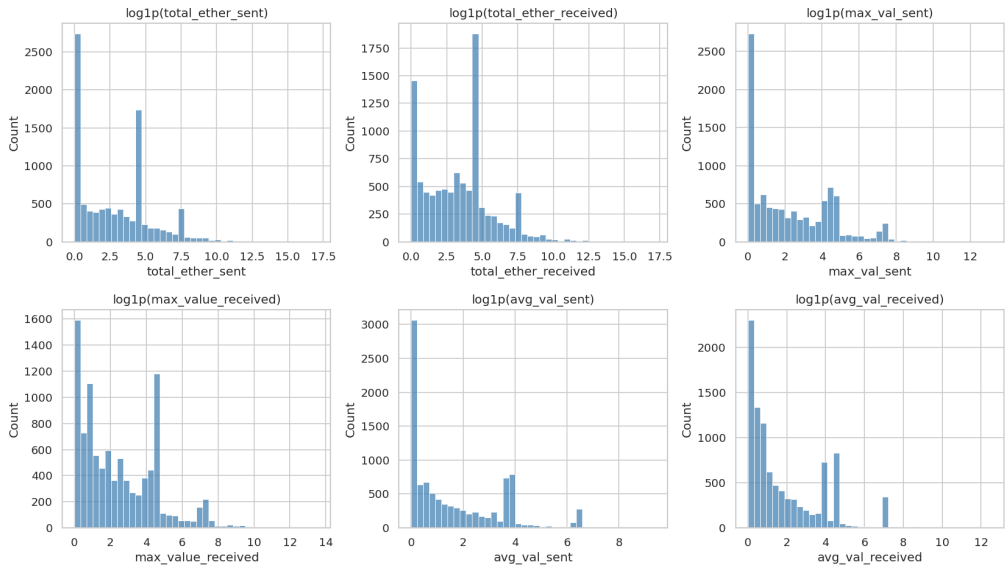


Figure 2.2. Distributions of the ether-volume features before and after the log transform

The analysis showed that 21 features are “sparse”: more than half of their values are zero. For the most part these are characteristics that simply do not apply to most accounts — for example, to addresses that never interacted with ERC20 tokens. This suggested adding boolean activity-indicator features that record the presence of such behaviour explicitly.

A separate data-quality problem was also found: 21 pairs of columns turned out to be bit-for-bit identical. These are mostly the groups of features related to the average time between ERC20 operations and to the volumes sent to contracts — all of which are zero for every row. Such columns are redundant, but importantly the coincidence is between features themselves, not between a feature and the target, so there is no direct leakage of the answer here. Correlation analysis (Fig. 2.3) confirmed the presence of groups of closely related features, but no feature correlates with the class label above 0.95, which likewise indicates the absence of obvious leakage.

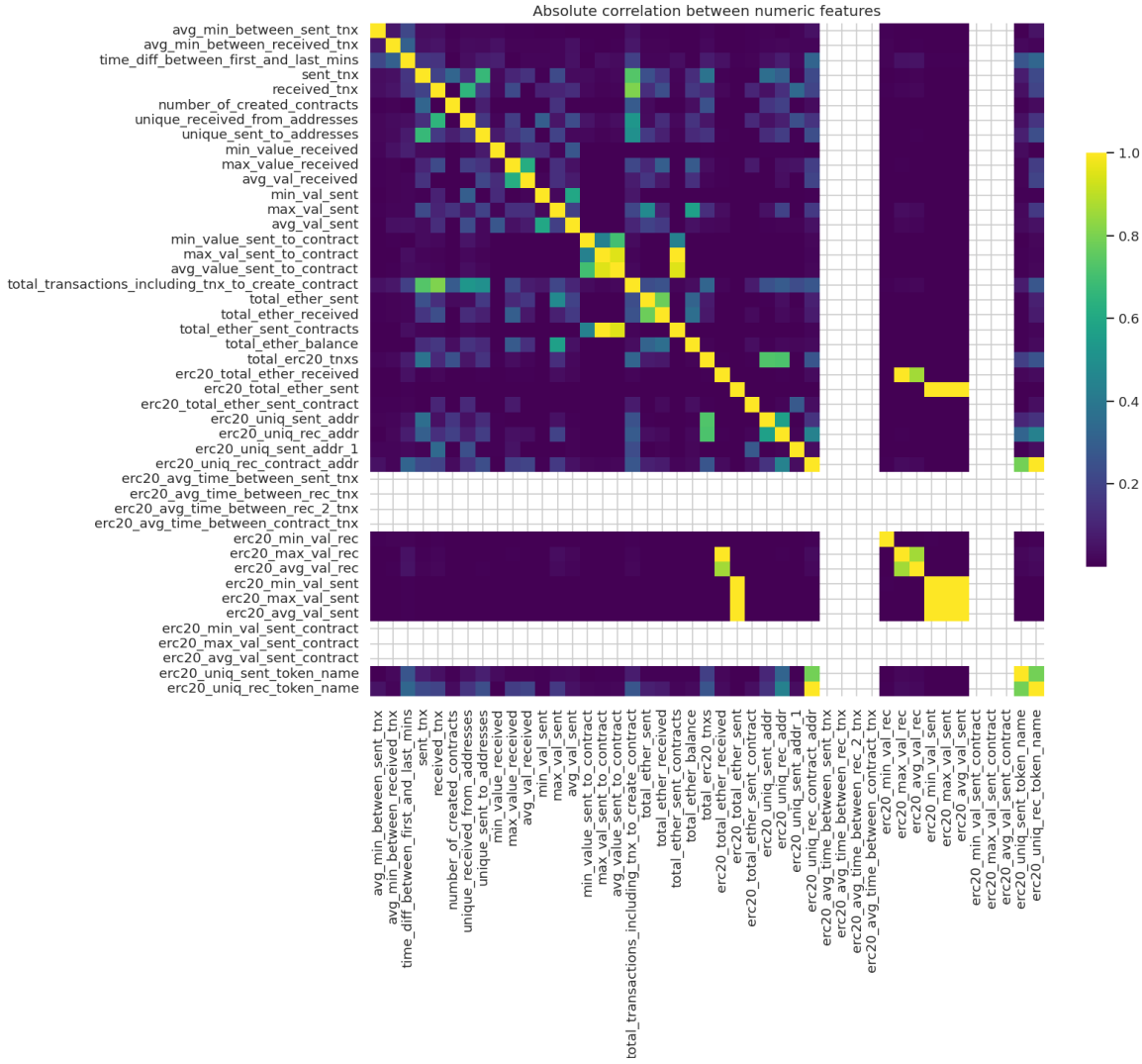


Figure 2.3. Heat map of correlations between features

To assess how well the features separate the classes, the absolute difference between the class medians divided by the standard deviation was computed for each of them [1]. The best separators are the span between the first and last operation, the total transaction counters, the numbers of sent and received transactions, the number of unique recipient addresses and the volume of ether received. The median values of these features by class are given in Table 2.1: fraudulent accounts live far shorter lives, carry out fewer and smaller operations with a narrower circle of counterparties, and the funds they receive arrive in denser bursts (a smaller interval between operations).

Table 2.1. Median values of the key features by class

Feature	Legitimate	Fraudulent
Lifetime of activity, days	81	5
Total transactions	11	5
Sent transactions	3	1
Received transactions	5	3
Unique recipient addresses	2	1
Interval between received operations, min	1402	82
Total ether received	71.72	1.68

The difference in the behaviour of the classes is clearly visible even by eye (Fig. 2.4): fraudulent accounts differ systematically from legitimate ones in transaction counts.

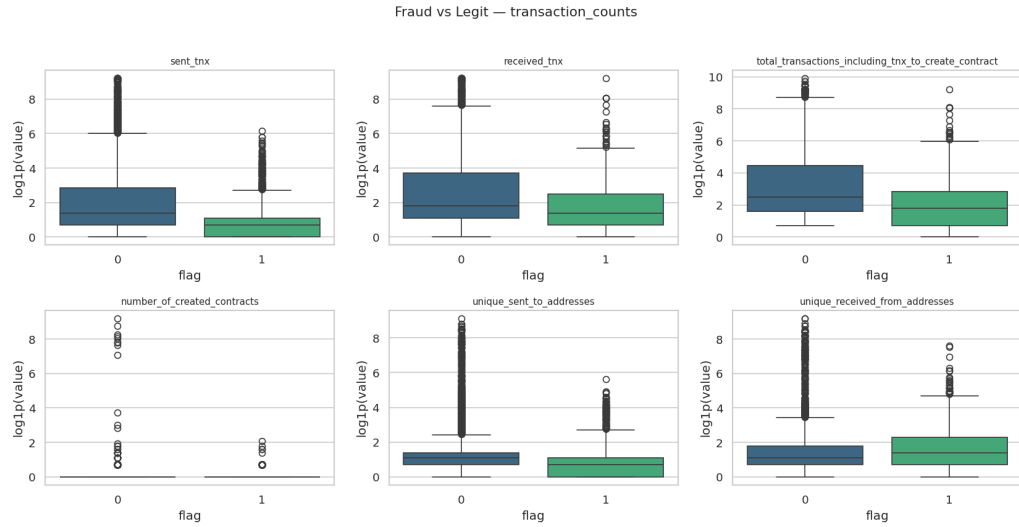


Figure 2.4. Transaction-count distributions for legitimate and fraudulent accounts

2.3 Duplicates and data leakage

The most important finding of the exploratory analysis was the duplicates. If the address column is discarded, the dataset contains 574 rows with completely identical feature values. They collapse into only 28 distinct profiles, and about

92% of them belong to the fraud class. The explanation is simple: these are fraudulent accounts with almost no activity, so their feature vectors coincide.

The duplicates are not an error in themselves — they are different real addresses. The problem appears at the splitting stage. Under an ordinary random split, exact copies of one profile are distributed into both the training and the test sample, after which the model is evaluated on rows it has already seen during training. To measure the scale of this effect, it was checked how many test rows have an exact copy in the training sample. Under a random stratified split there were 112 out of 1969 (about 5.7%), and almost all of them fraudulent.

To remove the problem, a grouped stratified split was used (the `grouped_stratified_split` function): rows with the same feature profile receive a common group identifier and go entirely to one side of the split, while the 78/22 class ratio is preserved. After this, no test row has a copy in the training sample.

Another result turned out to be interesting. One might have expected removing the leakage to lower the tree metrics noticeably, but in fact they barely changed: PR-AUC stayed at roughly 0.999 both before and after the fix. So contamination of the test sample was not the cause of the high scores — the features on their own separate fraudsters from legitimate accounts well. Fixing the split still matters, though: the numbers can now be trusted without wondering how much of them came from memorising familiar rows.

A caveat is in order: the test samples before and after the change of split consist of different rows, so this is a sanity check rather than a strictly controlled experiment. Still, the conclusion that there is no substantial drop in the metrics is clear enough.

2.4 Feature engineering and preprocessing

Before the models were trained, the raw data were brought into a convenient form. Column names were normalised to a single style, and the bookkeeping identifiers (the row index and the address) were removed, since they carry no

information about an account’s behaviour.

Based on the observations from the exploratory analysis, new features were added:

- log-transformed values of the six skewed ether-volume features;
- behavioural ratios — the ratio of ether sent to ether received, the net ether flow, the number of transactions per minute, the shares of unique counterparties among operations;
- boolean activity indicators — the presence of ERC20 operations, the fact of contract creation, a negative balance;
- the total number of an account’s unique counterparties.

Preprocessing is implemented as a `ColumnTransformer`-based pipeline that handles numeric and categorical features separately. For numeric features missing values are imputed with the median and the values are then standardised; for the two categorical features (the ERC20 token types) missing values are imputed with the most frequent value and the features themselves are one-hot encoded. Crucially, the pipeline is fitted exclusively on the training sample and saved in that form, after which it is applied to the test data. Because of this, statistics of the test sample do not seep into training, which is consistent with the requirements for correct evaluation set out in Section 1.5.

2.5 Model training and selection of the best model

The four models discussed in the first chapter were trained and compared: logistic regression, random forest, XGBoost and LightGBM. Each was given settings that account for class imbalance. The main hyperparameters are listed in Table 2.2.

Table 2.2. Main model hyperparameters

Model	Main hyperparameters
Logistic regression	<code>max_iter=1000</code> , <code>class_weight=balanced</code>
Random forest	300 trees, <code>max_depth=12</code> , <code>class_weight=balanced</code>
XGBoost	400 trees, <code>max_depth=6</code> , <code>learning_rate=0.05</code> , <code>scale_pos_weight=3.5</code>
LightGBM	400 trees, <code>num_leaves=63</code> , <code>learning_rate=0.05</code> , <code>is_unbalance</code>

The best model was chosen not by the results on the test set but by five-fold grouped stratified cross-validation on the training sample, averaging PR-AUC over the folds. Selecting on cross-validation rather than on the test set is a matter of principle: the test sample must remain independent and be used only for the final check. The cross-validation results are given in Table 2.3.

Table 2.3. Model quality by cross-validation (PR-AUC on the training sample)

Model	PR-AUC (mean $\pm$ std. dev.)
Logistic regression	0.913 ± 0.016
Random forest	0.995 ± 0.001
XGBoost	0.9958 ± 0.002
LightGBM	0.9959 ± 0.002

By the cross-validation results the best model was LightGBM, which was therefore chosen as the final one. It is worth noting that the three ensemble models differ by less than 0.001 in PR-AUC, that is, they are effectively equivalent; on a different split any of them could have come first. This is exactly why selection by cross-validation is more reliable than selection by a single result on the test set, which would amount to picking whichever model happened to be luckiest on that particular sample. In practice the final choice among these

three would be made on training speed, model size or ease of interpretation.

2.6 Test-set results and their analysis

The final quality estimate was obtained on the held-out test sample formed by the grouped split. The metric values for all models are given in Table 2.4.

Table 2.4. Model metrics on the test sample

Model	Precision	Recall	F_1	ROC-AUC	PR-AUC
Logistic regression	0.719	0.940	0.815	0.982	0.947
Random forest	0.979	0.986	0.983	0.9996	0.9987
XGBoost	0.982	0.986	0.984	0.9998	0.9992
LightGBM	0.986	0.989	0.987	0.9998	0.9994

The ROC and precision–recall curves for all models are shown in Fig. 2.5 and Fig. 2.6. They make the advantage of the ensemble methods plain: their curves practically merge in the upper corner, while logistic regression is noticeably behind.

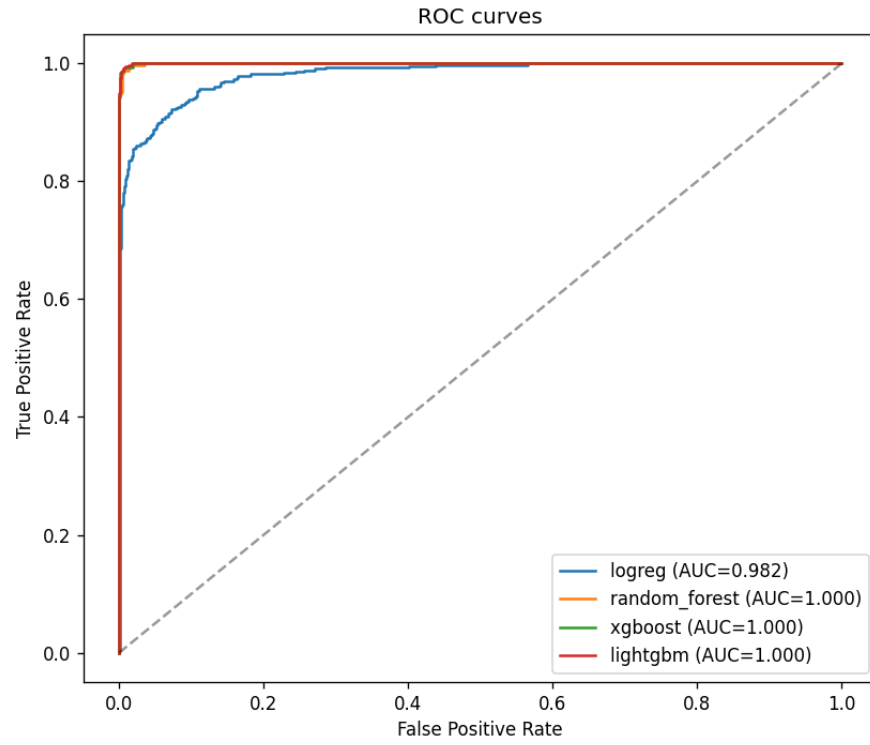


Figure 2.5. ROC curves of the models on the test sample

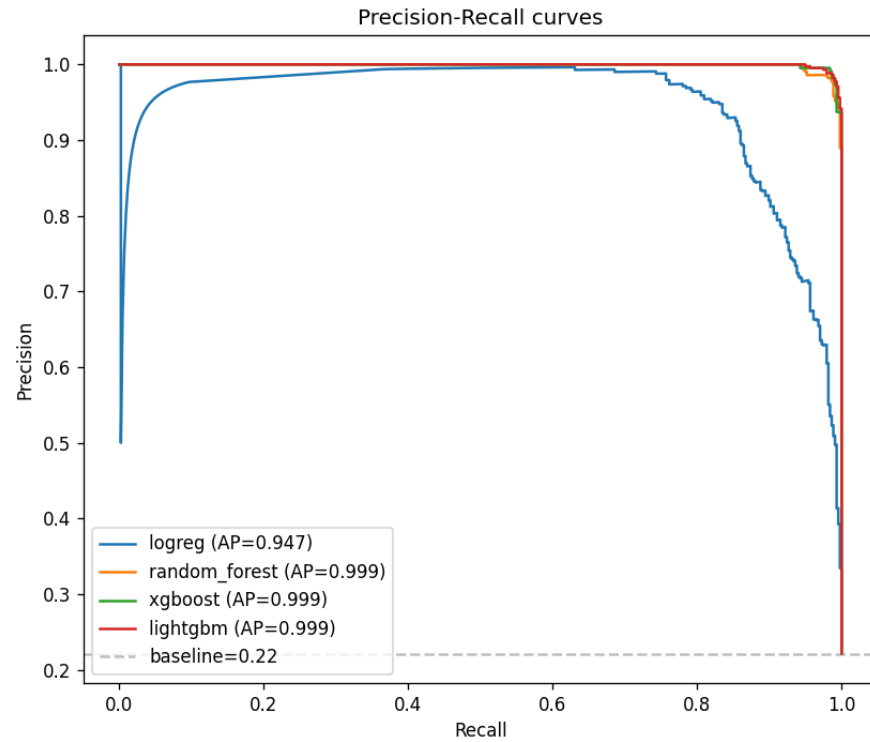


Figure 2.6. Precision-recall curves of the models on the test sample

The confusion matrix of the final LightGBM model (Fig. 2.7) shows that the overwhelming majority of accounts are classified correctly, and the numbers of missed fraudsters and false alarms are small.

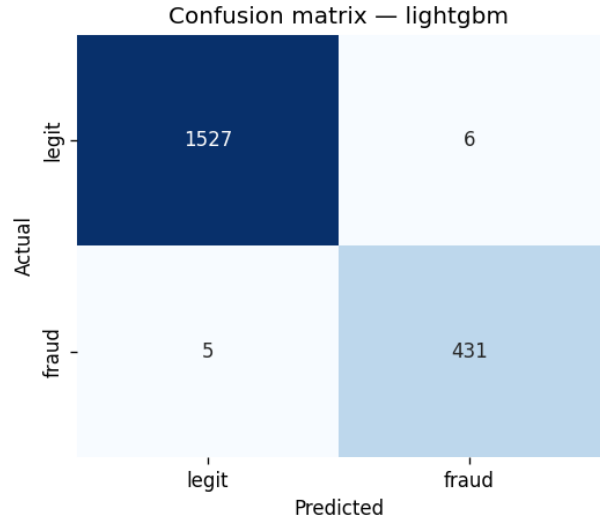


Figure 2.7. Confusion matrix of the LightGBM model

The feature-importance analysis of the final model (Fig. 2.8) agrees with the conclusions of the exploratory analysis: among the most influential features are the account’s activity time span, the transaction counters and the number of unique counterparties — exactly the features that separated the classes best.

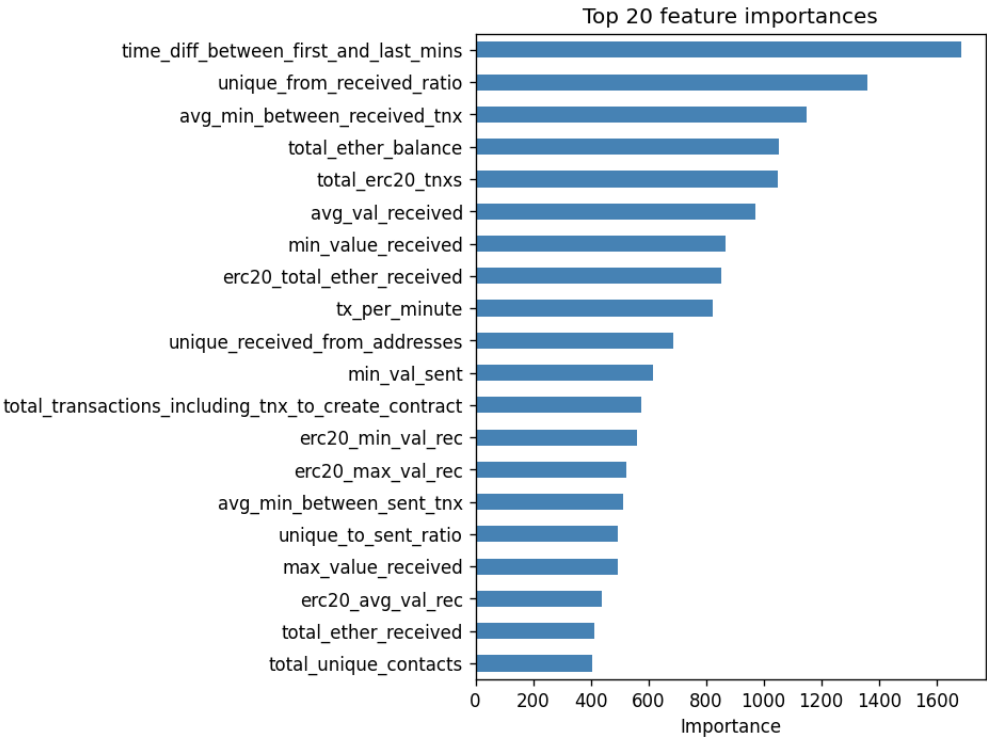


Figure 2.8. The twenty most influential features of the final model

Several observations follow from these results. First, tree ensembles have a clear advantage: all three reach a PR-AUC of at least 0.998, whereas logistic

regression stops at around 0.947. This points to substantial non-linear structure in the data that a linear model cannot capture. Second, logistic regression sacrifices precision for recall: it finds about 94% of fraudsters, but in almost every third case it is wrong, labelling a legitimate account as fraudulent — a consequence of the class weighting, which pushes the model towards detecting positives. Third, in this problem it is recall that matters most: even the best model misses a few fraudulent accounts (recall 0.989, that is, roughly 5 out of the 435 fraudsters in the test sample), and it is that number of misses that would be watched in a real monitoring system.

CONCLUSIONS

This work addresses the automatic detection of fraudulent Ethereum accounts, stated as a binary classification problem over behavioural features of transaction activity. Exploratory data analysis was carried out, additional features were engineered, a preprocessing pipeline was built, and four machine-learning models were trained and compared. The dataset covered 9841 accounts described by fifty features of transaction activity, with a fraud share of about 22%. The exploratory analysis additionally revealed problems in the quality of the source data: groups of duplicate features and a large share of zero values, both of which were taken into account during feature engineering.

The most important methodological result was the detection and removal of data leakage. The dataset contains 574 rows with identical feature profiles, collapsing into 28 distinct profiles, mostly fraudulent. It was shown that under an ordinary random split about 5.7% of the test rows have an exact copy in the training sample, that is, the model is evaluated on what it has already seen. To remove this, a grouped stratified split was applied, after which no test row repeats a training one and the class ratio is preserved.

The comparison of models confirmed the expected advantage of tree ensembles over the linear model: all three ensemble methods reached a PR-AUC of at least 0.998, whereas logistic regression reached about 0.947. The best model by cross-validation was LightGBM, which was chosen as the final one (PR-AUC about 0.999 on the test sample). PR-AUC was justifiably taken as the main metric, since under class imbalance it is more informative than accuracy and than ROC-AUC. On the held-out test sample the final model achieved a precision of 0.986, a recall of 0.989 and a PR-AUC of about 0.999: of all the fraudulent accounts it missed only five, and the number of false alarms was six, which indicates that the classifier built here is of high practical usefulness.

One unexpected observation deserves a separate mention: removing the leakage had almost no effect on the metric values. This means that the high scores were produced not by contamination of the test sample but by the genuine separating power of the features. Fixing the split nevertheless remains essential, since it is what makes the results trustworthy.

The practical value of the work lies in the fraud-detection pipeline that was built and, above all, in the approach to splitting the data correctly, which can be applied to similar problems with duplicates. The pipeline is reproducible: the saved preprocessor and the trained model make it possible to classify new accounts without retraining, and the grouped-split approach itself transfers easily to other datasets with repeated records.

The limitations of the study should also be noted. The conclusion about the effect of leakage is based on a comparison of samples consisting of different rows, so it is a sanity check rather than a strictly controlled experiment. In addition, the results were obtained on a single dataset, so generalising them to other networks requires further verification.

Promising directions for further work are hyperparameter tuning by random search or optimisation methods, tuning the decision threshold to account for the different costs of a missed fraudster and a false alarm, applying prediction-interpretation methods (for example SHAP) to explain individual model decisions, and integrating the classifier into an online monitoring system that flags suspicious accounts in real time.

BIBLIOGRAPHY

1. Yarovyi A. T., Strakhov Ye. M. *Multivariate Statistical Analysis: a course-book for students of mathematical and economic disciplines*. Odesa: Astro-print, 2015. 132 p.
2. Aliyev V. Ethereum Fraud Detection Dataset. *Kaggle*. URL: <https://www.kaggle.com/datasets/vagifa/ethereum-frauddetection-dataset>.
3. Breiman L. Random Forests. *Machine Learning*. 2001. Vol. 45, No. 1. P. 5–32.
4. Buterin V. Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform: white paper. 2014. URL: <https://ethereum.org/en/whitepaper/>.
5. Chawla N. V., Bowyer K. W., Hall L. O., Kegelmeyer W. P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*. 2002. Vol. 16. P. 321–357.
6. Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. New York: ACM, 2016. P. 785–794.
7. Davis J., Goadrich M. The Relationship Between Precision-Recall and ROC Curves. *Proceedings of the 23rd International Conference on Machine Learning*. New York: ACM, 2006. P. 233–240.
8. Farrugia S., Ellul J., Azzopardi G. Detection of Illicit Accounts over the Ethereum Blockchain. *Expert Systems with Applications*. 2020. Vol. 150. Article 113318.
9. Friedman J. H. Greedy Function Approximation: A Gradient Boosting Machine. *The Annals of Statistics*. 2001. Vol. 29, No. 5. P. 1189–1232.
10. Hastie T., Tibshirani R., Friedman J. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. New York: Springer, 2009. 745 p.
11. He H., Garcia E. A. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*. 2009. Vol. 21, No. 9. P. 1263–1284.
12. Kaufman S., Rosset S., Perlich C., Stitelman O. Leakage in Data Mining: Formulation, Detection, and Avoidance. *ACM Transactions on Knowledge*

- Discovery from Data*. 2012. Vol. 6, No. 4. Article 15.
13. Ke G., Meng Q., Finley T., Wang T., Chen W., Ma W., Ye Q., Liu T.-Y. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. *Advances in Neural Information Processing Systems* 30. 2017. P. 3146–3154.
 14. McKinney W. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. 2nd ed. Sebastopol: O'Reilly Media, 2017. 544 p.
 15. Mitchell T. M. *Machine Learning*. New York: McGraw-Hill, 1997. 414 p.
 16. Müller A. C., Guido S. *Introduction to Machine Learning with Python: A Guide for Data Scientists*. Sebastopol: O'Reilly Media, 2016. 400 p.
 17. Pedregosa F., Varoquaux G., Gramfort A. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*. 2011. Vol. 12. P. 2825–2830.
 18. Powers D. M. W. Evaluation: From Precision, Recall and F-Measure to ROC, Informedness, Markedness and Correlation. *Journal of Machine Learning Technologies*. 2011. Vol. 2, No. 1. P. 37–63.
 19. Saito T., Rehmsmeier M. The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets. *PLOS ONE*. 2015. Vol. 10, No. 3. e0118432.

APPENDIX A

Implementation of the grouped stratified data split

The Python code below implements the grouped stratified split described in Section 2.3. Rows with the same feature profile are assigned a common group identifier, after which the sample is split so that no group lands in both the training and the test part, while the class ratio is preserved.

```
def _profile_group_ids(df, target):
    # rows with the same feature values get the same group id
    feat = df.drop(columns=[target])
    row_hash = pd.util.hash_pandas_object(feat, index=False)
    return pd.Series(pd.factorize(row_hash)[0], index=df.index)

def grouped_stratified_split(df, target, test_size, seed):
    # group identical rows, keep each group on one side,
    # and preserve the fraud/legit ratio
    n_splits = max(2, round(1 / test_size))
    groups = _profile_group_ids(df, target)
    sgkf = StratifiedGroupKFold(
        n_splits=n_splits, shuffle=True, random_state=seed
    )
    train_idx, test_idx = next(sgkf.split(df, df[target], groups))
    train = df.iloc[train_idx].reset_index(drop=True)
    test = df.iloc[test_idx].reset_index(drop=True)
    return train, test
```